

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENTTOOL3- DEBUGGING & OPTIMISATIONTASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	I Raiyan	Reg.No.:	192521421
Department:	IT	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I K. A R I S T O T L E (1 9 2 5 1 1 1 9 6) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Aristotle

MARKS ALLOCATION

	Identification of Errors / Bugs(20%)	Debugging Approach & Analysis(20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied(20%)	Testing and Documentation(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

			Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output
			Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality
Criteria		Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors: misses key issues	Unable to identify errors or performs complete testing with proper validation and clear documentation
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with	Uses basic debugging methods with limited analysis	No systematic debugging approach followed

Question 1

Problem Statement:

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Q1. Debugging Signed Integer Addition Using 2's Complement

1. Problem Identification

The program performs signed integer addition using **8-bit 2's complement representation**. When adding two large positive numbers such as **+120** and **+50**, the output becomes incorrect because the actual result exceeds the range that can be represented using 8 bits.

8-bit signed integer range:

- Minimum = **-128**
- Maximum = **+127**

Expected result:

120 + 50 = 170

Since **170 > 127**, it cannot be represented in an 8-bit signed number. Therefore, **overflow occurs**.

2. Error / Bug Identification

Step 1: Convert numbers into binary

+120

120 = **01111000**

+50

50 = **00110010**

Step 2: Perform binary addition

01111000
+ 00110010

10101010

Binary result:

10101010

In 2's complement, this represents:

Invert bits:

01010101

Add 1:

01010110 = 86

Therefore = -86

The processor gives **-86** instead of **170**, which is incorrect.

Error Identified

The addition itself is correct, but the **result exceeds the representable range**, causing **signed overflow**.

3. Debugging Approach & Analysis

Overflow in signed 2's complement arithmetic occurs when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Here,

120 → Positive

50 → Positive

Result = Negative

Hence,

Overflow has occurred.

Overflow Detection Rule

Overflow occurs if:

Carry into MSB ≠ Carry out of MSB

OR

(Sign of Operand A == Sign of Operand B)

AND

(Sign of Result ≠ Sign of Operands)

Here,

Operand A Sign = 0

Operand B Sign = 0

Result Sign = 1

Since the signs are different,

Overflow = TRUE

4. Correct Debugged Solution

The program should not simply display the binary result. Instead, it should first check for overflow.

Algorithm

Read two signed integers

Convert to 8-bit binary

Perform addition

If

(SignA == SignB) AND (SignResult != SignA)

Then

Display

"Overflow Detected"

Else

Display Result

Correct Output

Input 1 = 120

Input 2 = 50

Result = Overflow Detected

Actual Sum = 170

Cannot be represented in 8-bit signed integer.

5. Optimization Technique

To improve the program:

Method 1: Overflow Detection Logic

Use XOR gates to compare carry into and carry out of the Most Significant Bit (MSB). This detects overflow immediately without extra computation.

Method 2: Use Larger Data Type

Instead of 8-bit,

use

- 16-bit
- 32-bit
- 64-bit integers

This increases the range and avoids overflow for larger values.

Method 3: Exception Handling

When overflow occurs,

- stop incorrect computation,
- display an error message,
- or promote the result to a larger integer type.

6. Testing & Validation

Test Case Input A Input B Expected Output Result

1 20 30 50 Pass

2 100 20 120 Pass

3 120 50 Overflow Detected Pass

4 -100 -50 Overflow Detected Pass

5 -40 20 -20 Pass

Validation

- Positive + Positive within range → Correct result.
- Negative + Negative within range → Correct result.
- Positive + Positive exceeding +127 → Overflow detected.

- Negative + Negative below $-128 \rightarrow$ Overflow detected.

7. Conclusion

The incorrect result occurs because the sum of **+120** and **+50** exceeds the **8-bit signed integer range (-128 to +127)**. During addition, the binary result becomes **10101010**, which is interpreted as **-86** due to 2's complement representation. The bug is fixed by detecting signed overflow using the sign-bit rule or carry comparison before displaying the result. If overflow is detected, the program should report **"Overflow Detected"** or use a larger integer type (16-bit/32-bit) to represent the correct value. This ensures accurate computation and reliable program execution.

Q2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

1. Problem Identification

The processor uses a **Ripple Carry Adder (RCA)** for arithmetic operations. During addition, the carry generated at one bit must propagate through every subsequent bit before the final result is produced. In a real time system, this sequential carry propagation causes a significant execution delay, reducing system performance.

2. Error/Bug Identification

Identified Issue:

- The carry output from each bit depends on the carry input of the previous bit.
- The carry propagates one stage at a time, creating a long propagation delay.
- As the number of bits increases (8-bit, 16-bit, 32-bit, etc.), the total delay also increases.

Example (4-bit Addition)

A = 1011
B = 0101

Bit0 $\rightarrow$ Carry to Bit1

Bit1 $\rightarrow$ Carry to Bit2

Bit2 $\rightarrow$ Carry to Bit3

Bit3 $\rightarrow$ Final Carry

Each bit waits for the previous carry before computing its sum.

3. Debugging Approach & Analysis

Root Cause Analysis

The bottleneck occurs because the Ripple Carry Adder performs carry propagation sequentially. Carry equations:

C1 depends on C0

C2 depends on C1

C3 depends on C2

C4 depends on C3

Therefore,

Total Delay = Delay(Bit0)

+ Delay(Bit1)

+ Delay(Bit2)

+ Delay(Bit3)

Since every carry waits for the previous carry, the delay increases linearly with the number of bits. **Conclusion:** The major performance bottleneck is the slow carry propagation.

4. Correct Debugged Solution

Replace the **Ripple Carry Adder (RCA)** with a **Carry Look-Ahead Adder (CLA)**. The CLA computes carries in parallel using:

Generate Function

$G_i = A_i \times B_i$

Propagate Function

$P_i = A_i \oplus B_i$

Carry equations:

$C_1 = G_0 + P_0C_0$

$$C2 = G1 + P1G0 + P1P0C0$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

$$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0$$

Since all carries are calculated simultaneously, there is no need to wait for the previous carry.

5. Optimization Technique Applied

Carry Look-Ahead Adder (CLA)

Advantages

- Calculates carries simultaneously.
- Greatly reduces propagation delay.
- Faster arithmetic operations.
- Suitable for real-time applications.
- Improves processor performance.

Comparison

Ripple Carry Adder Carry Look-Ahead Adder

Carry propagates sequentially Carry generated in parallel

High delay Very low delay

Simple design Slightly complex design

Slower for large bit sizes Faster for large bit sizes

Less suitable for real-time systems Ideal for real-time systems

6. Testing & Validation

Test Case 1

A = 1010

B = 0101

Expected Output

Sum = 1111

Both RCA and CLA produce the same correct result.

Test Case 2

A = 1111

B = 0001

Expected Output

Sum = 0000

Carry = 1

The CLA produces the correct result while completing the operation much faster due to parallel carry computation.

Validation

- ✓ Correct sum obtained.
- ✓ Carry generated correctly.
- ✓ Delay significantly reduced.
- ✓ Suitable for high-speed processors and real-time systems.

7. Conclusion

The performance bottleneck in the Ripple Carry Adder is caused by **sequential carry propagation**, where each carry must wait for the previous one. Replacing it with a **Carry Look-Ahead Adder (CLA)** eliminates this bottleneck by computing carry signals in parallel using **Generate (G)** and **Propagate (P)** logic. This significantly reduces computation time

Question 3

A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure

accurate signed multiplication.

Answer (Level-4 Rubric)

1. Problem Identification

The multiplication unit using **Booth's Algorithm** produces incorrect results when multiplying **negative signed numbers**. The error occurs due to mistakes in:

- Bit-pair checking (Q0 and Q-1)
- Arithmetic right shifting
- Sign extension during shifting

These errors lead to an incorrect final product.

2. Identification of Errors/Bugs

The possible bugs are:

Error 1: Incorrect Bit-Pair Checking

Booth's Algorithm examines the pair (Q0, Q-1).

Correct rules are:

Q0 Q-1 Operation

00 No operation

01 $A = A + M$

10 $A = A - M$

11 No operation

Bug:

The processor performs the wrong arithmetic operation for the detected bit pair.

Error 2: Incorrect Arithmetic Right Shift

After every operation, the registers must undergo an **Arithmetic Right Shift (ARS)**. Bug:

- Logical shift is used instead of arithmetic shift.
- MSB becomes 0 instead of preserving the sign bit.

Result:

Negative numbers become incorrect.

Error 3: Incorrect Sign Extension

When shifting a negative number, the MSB must remain 1.

Bug:

The implementation inserts 0 into the MSB.

This changes the negative value into an incorrect positive value.

3. Debugging Approach & Analysis

Step 1: Verify Bit-Pair Checking

Check every iteration.

Example:

If

$Q0 = 1$

$Q-1 = 0$

Operation should be

$A = A - M$

If the processor performs addition instead, the multiplication result becomes incorrect.

Step 2: Verify Arithmetic Shift

After addition/subtraction,

perform

Arithmetic Right Shift

Example:

Before shift

$A = 11110000$

Correct shift

11111000

Incorrect logical shift

01111000

The sign changes, producing wrong results.

Step 3: Verify Sign Extension

For every negative value,
the MSB should remain 1 after shifting.

Example

Before

11101010

After arithmetic shift

11110101

Incorrect result using logical shift

01110101

Hence the sign information is lost.

4. Correct Debugged Solution

Implement Booth's Algorithm correctly.

Algorithm

1. Initialize registers A, Q, M and Q-1.
2. Check pair (Q₀, Q₋₁).
3. Perform:
 - 01 → A = A + M
 - 10 → A = A - M
 - 00 or 11 → No operation
4. Perform Arithmetic Right Shift.
5. Repeat for n bits.
6. Store final result in AQ.

This ensures correct multiplication for both positive and negative integers.

5. Optimization Techniques Applied

To improve performance:

Correct Arithmetic Right Shift

Use arithmetic shift instead of logical shift to preserve the sign bit.

Proper Sign Extension

Always copy the MSB during shifting.

This maintains correct signed representation.

Accurate Bit-Pair Detection

Implement hardware logic that correctly identifies
(Q₀, Q₋₁)

to avoid incorrect addition or subtraction.

Efficient Control Logic

Reduce unnecessary operations by executing only the required arithmetic operation based on the bit pair. This improves execution speed and accuracy.

6. Testing & Validation

Test the multiplication unit with different cases.

Test Case Expected Result Status

+5 × +3 = 15 Correct

-5 × +3 = -15 Correct

+5 × -3 = -15 Correct

-5 × -3 = 15 Correct

-8 × -2 = 16 Correct

Validation confirms that:

- Correct bit-pair checking is performed.
- Arithmetic shifts preserve the sign.
- Sign extension is accurate.
- All positive and negative multiplications produce correct results.

7. Conclusion

The incorrect results in Booth's multiplication algorithm are mainly caused by **wrong bit-pair checking, improper arithmetic right shifting, and incorrect sign extension**. By correcting these issues and ensuring proper implementation of Booth's Algorithm, the multiplication unit produces accurate signed multiplication results while improving reliability and performance.

Q4. Debug the Restoring Division Algorithm and Optimize Using the Non-Restoring Division Method (Level 4 Answer)

Problem Identification

An embedded system implementing the **Restoring Division Algorithm** experiences poor performance because the algorithm repeatedly restores the previous value whenever the subtraction result becomes negative. These extra restoration steps increase the execution time and reduce the efficiency of the system.

1. Identification of Errors / Bugs

Error Identified

- The restoring division algorithm performs an unnecessary **restoration** whenever the partial remainder becomes negative.
- This results in:
 - Additional arithmetic operations.
 - Increased execution time.
 - Lower processor efficiency.
 - Reduced performance in real-time embedded systems.

Root Cause

After every subtraction:

- If the remainder is **positive**, the quotient bit is set to **1**.
- If the remainder is **negative**, the algorithm restores the previous remainder by adding the divisor again before continuing.

The repeated restore operation is the main performance bottleneck.

2. Debugging Approach and Analysis

Step 1: Analyze the Workflow

Restoring Division Algorithm:

1. Initialize Dividend, Divisor, Quotient, and Remainder.
2. Shift left the remainder and quotient.
3. Subtract divisor from remainder.
4. Check the result.
 - If positive:
 - Set quotient bit = 1
 - If negative:
 - Restore the remainder by adding the divisor.
 - Set quotient bit = 0
5. Repeat until all bits are processed.

Problem Analysis

Whenever the subtraction result is negative, the algorithm performs:

Subtraction → **Negative Result** → **Restoration** → **Continue**

This extra restoration is repeated many times, causing unnecessary computations.

3. Correct Debugged Solution

Instead of using the restoring division algorithm, use the **Non-Restoring Division Algorithm**. **Working Principle**

1. Shift left the partial remainder.
2. Perform subtraction if the previous remainder is positive.
3. Perform addition if the previous remainder is negative.
4. Do **not restore** the previous remainder.
5. Set the quotient bit according to the sign of the remainder.
6. Perform one final correction if the remainder is negative.

Advantages

- Eliminates repeated restoration.
- Reduces arithmetic operations.
- Improves execution speed.
- Suitable for high-speed embedded processors.

4. Optimization Technique Applied

Optimization Used

Replace the **Restoring Division Algorithm** with the **Non-Restoring Division Algorithm**. **Restoring Division** **Non-Restoring Division**

Restores negative remainder	No restoration required
More arithmetic operations	Fewer arithmetic operations
Slower execution	Faster execution

Higher power consumption Lower power consumption

Less efficient More efficient

Performance Improvement

- Fewer addition operations.
- Reduced processor cycles.
- Faster division.
- Better utilization of CPU resources.
- Improved real-time performance.

5. Testing and Validation

Test Case

Dividend = 13

Divisor = 3

Expected Output:

- Quotient = 4
- Remainder = 1

Validation

Parameter Expected Obtained

Quotient 4 4

Remainder 1 1

Execution Time Lower Lower

Accuracy Correct Correct

The optimized Non-Restoring Division Algorithm produces the correct quotient and remainder while executing faster due to the elimination of repeated restoration steps.

Conclusion

The inefficiency in the restoring division algorithm is caused by repeated restoration whenever the partial remainder becomes negative. By replacing it with the **Non-Restoring Division Algorithm**, these unnecessary operations are eliminated, resulting in faster execution, fewer arithmetic operations, lower power consumption, and improved performance in embedded systems while maintaining accurate division results. This optimized approach is well suited for real-time applications.

Question 5

A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Answer (Based on Level-4 Rubric)

1. Problem Identification

The application uses **IEEE 754 single-precision (32-bit)** floating-point representation. When a **very small number** is added to a **very large number**, the result becomes inaccurate because the small value may be lost during floating-point computation.

Example

Large number = **1,000,000.0**

Small number = **0.000001**

Expected Result:

1,000,000.000001

Actual Result (Single Precision):

1,000,000.0

The fractional part is lost due to limited precision.

2. Error/Bug Identification

The inaccuracy occurs because of three major reasons:

(a) Exponent Alignment

Before addition, IEEE 754 aligns the exponents of both numbers.

- The number with the smaller exponent shifts its mantissa to the right.
- If the exponent difference is large, many significant bits are discarded.
- As a result, the smaller number becomes nearly zero.

Bug Identified:

Loss of precision during exponent alignment.

(b) Normalization

After addition, the result is normalized.
If normalization shifts bits again, additional precision may be lost.

Bug Identified:

Normalization introduces further rounding errors.

(c) Rounding Error

IEEE 754 single precision stores only **23 fraction bits**.
Extra bits are rounded according to IEEE 754 rounding rules.
This rounding may remove important decimal information.

Bug Identified:

Limited mantissa causes rounding errors.

3. Debugging Approach and Analysis

Step 1

Check the binary representation of both operands.

Verify

- Sign bit
- Exponent
- Mantissa

Step 2

Observe exponent alignment.
If the exponent difference is very large,
Small Number Mantissa

↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓

000000000000000000000001

↓

Shift Right

↓

000000000000000000000000

The small value disappears.

Step 3

Monitor normalization.
Ensure normalization is performed correctly without unnecessary precision loss.

Step 4

Check rounding mode.
IEEE 754 usually uses

Round to Nearest Even

Improper rounding increases error.

Root Cause

The main cause is the **limited precision of single-precision floating-point representation**, especially during exponent alignment and rounding.

4. Correct Debugged Solution

Solution 1 – Use Double Precision (64-bit)

Double precision provides

- 52-bit mantissa
- Higher precision
- Better accuracy
- Smaller rounding error

Example

Single Precision

1000000.0 + 0.000001

↓

1000000.0

Double Precision
 $1000000.0 + 0.000001$

↓

1000000.000001
The smaller value is preserved.

Solution 2 – Rearrange Arithmetic Operations

Instead of
Large + Small
Accumulate small values first.

Example
Instead of
 $1000000 + 0.000001 + 0.000002$
Use
 $(0.000001 + 0.000002)$

↓

0.000003

↓

$1000000 + 0.000003$
This reduces precision loss.

Solution 3 – Use Numerically Stable Algorithms

Algorithms such as **Kahan Summation** compensate for floating-point errors by tracking lost low-order bits, improving the accuracy of repeated additions.

5. Optimization Techniques Applied

To improve performance and accuracy:

- Use **IEEE 754 double precision (64-bit)** instead of single precision.
- Minimize the addition of numbers with very different magnitudes.
- Accumulate small values before adding them to large values.
- Use **Kahan Summation** or other numerically stable summation methods.
- Reduce unnecessary normalization and repeated rounding operations.

These techniques improve computational accuracy while maintaining reliable floating-point calculations.

6. Testing and Validation

Test Case 1

Input
 $1000000.0 + 0.000001$
Expected Output
 1000000.000001
Result

- Single Precision → Inaccurate
- Double Precision → Accurate

Test Case 2

Input
 $1000.5 + 0.25$
Expected Output
 1000.75
Result

Correct in both single and double precision because the exponent difference is small. **Test Case 3**

Input
 $0.1 + 0.2$
Expected Output
Approximately
 0.30000000000000004

This demonstrates the normal limitations of binary floating-point representation, which can be reduced with higher precision.

7. Conclusion

The inaccuracies occur due to **exponent alignment, normalization, and rounding** in IEEE 754 single precision floating-point arithmetic. The most effective solution is to use **double precision**, apply **numerically stable algorithms** such as **Kahan Summation**, and organize arithmetic operations to minimize precision loss. These optimizations improve accuracy and ensure reliable floating-point computations.